UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO



FACULTAD DE INGENIERÍA



MODELADOS DE PROGRAMACIÓN ORIENTADA A OBJETOS

REPORTE DE CÓDIGO: PRÁCTICA 7

ALUMNO: MIGUEL ANGEL MEDINA REYES

NÚMERO DE CUENTA: 322287123

PROFESOR: OSCAR MANUEL RUIZ HURTADO

SEMESTRE: 2027-1

1. Tipo de Código Utilizado

El código desarrollado en esta práctica pertenece al paradigma de la **Programación Orientada a Objetos (POO)** y está escrito en el lenguaje **Java**. En este código se aplican los siguientes conceptos y pilares fundamentales:

- **Herencia (extends):** Es un mecanismo que permite crear nuevas clases (llamadas clases hijas o subclases) a partir de clases ya existentes (clases padre o superclases). Al heredar, la clase hija adquiere automáticamente todos los atributos y métodos de la clase padre, estableciendo una relación de tipo "es un" (por ejemplo, un Gato *es un* Animal). Esto promueve la reutilización de código y facilita la creación de jerarquías lógicas.

- **Encapsulamiento (private y getters):** Es el principio de ocultar los detalles internos y los datos de un objeto del mundo exterior. Se logra declarando los atributos como privados (private) para que ninguna otra clase pueda modificarlos directamente. En su lugar, la clase proporciona métodos públicos específicos, conocidos como *getters* (para leer el dato) y *setters* (para modificarlo), lo que permite validar la información y proteger la integridad del objeto (la "cápsula").

- **Constructores y Constructores en Cadena (super(...)):** Un constructor es un bloque de código especial que se ejecuta automáticamente cuando se crea (instancia) un nuevo objeto de la clase, y sirve para inicializar sus atributos. En jerarquías de herencia, se utilizan los **constructores en cadena**: la primera instrucción en el constructor de una clase hija debe ser obligatoriamente la llamada al constructor de su padre usando super(...). Esto garantiza que la parte "padre" del objeto se construya correctamente antes de añadirle las características específicas del hijo.

- **Sobrescritura de métodos (@Override):** Es la capacidad que tiene una clase hija de redefinir o modificar el comportamiento de un método que heredó de su clase padre. En Java, se utiliza la etiqueta @Override para indicarle al compilador que se está reemplazando la implementación original. Esto permite que objetos de diferentes clases hijas respondan de manera distinta al mismo método (por ejemplo, que el toString() de Perro devuelva algo diferente al toString() de Personaje).

2. Explicación de las Clases: Parte 1 (Animales)

Esta jerarquía simula diferentes tipos de animales domésticos y cómo heredan características.

- **Animal.java (Clase Padre / Base):**
    - **Qué hace:** Define las características más básicas de un animal: nombre y edad. En su constructor implementa una validación lógica (cápsula) para evitar que se guarden edades negativas (si la edad es menor a 0, la guarda como 0). Define un sonido genérico "...".
- **Perro.java (Clase Hija de Animal):**

    - **Qué hace:** Hereda de Animal y añade el atributo específico raza. Modifica el método sonido() para que devuelva "Guau" y reutiliza el método toString() de su padre agregando la raza.
- **PerroGuia.java (Clase Hija de Perro):**
    - **Qué hace:** Es el tercer nivel de la jerarquía. Hereda de Perro (y por lo tanto también de Animal). Añade a quién asiste el perro. **No sobrescribe** el método sonido(), por lo que automáticamente hereda el "Guau" del Perro normal, demostrando la reutilización de código.
- **Gato.java (Clase Hija de Animal):**
    - **Qué hace:** Hereda directamente de Animal. Añade un valor booleano para saber si es un gato de interior o exterior. Define su propio sonido como "Miau".
- **MainAnimales.java (Clase Principal / Piloto):**
    - **Qué hace:** Contiene el método main que arranca el programa. Su función es instanciar (crear objetos) de las clases anteriores y mandar a llamar sus métodos en consola para comprobar que la herencia y las validaciones funcionen correctamente.

3. Explicación de las Clases: Parte 2 (Personajes)

Esta jerarquía simula un sistema de clases para un videojuego de rol (RPG), enfocándose en cómo se calcula el daño de ataque en cadena.

- **Personaje.java (Clase Padre / Base):**
    - **Qué hace:** Define el nombre y la vida de cualquier personaje. Protege la variable de vida asegurándose de que nunca baje de 0 ni suba de 100. Define un ataque base de 10 puntos.
- **Caballero.java (Clase Hija de Personaje):**
    - **Qué hace:** Añade el atributo fuerza. En su método atacar(), no reinventa el ataque base, sino que manda a llamar al ataque del padre (super.atacar()) y le suma su fuerza.
- **Paladin.java (Clase Hija de Caballero):**
    - **Qué hace:** Añade el atributo fe. Representa una cadena profunda, ya que su método atacar() llama a super.atacar() (que invoca al del Caballero, que a su vez invoca al del Personaje) y le suma la mitad de su fe, logrando un cálculo escalonado.
- **Mago.java (Clase Hija de Personaje):**
    - **Qué hace:** Añade el atributo mana. A diferencia del Caballero, el Mago sobrescribe su método atacar() de forma independiente, ignorando el ataque base del padre y calculando su daño basándose únicamente en la mitad de su maná.
- **MainPersonajes.java (Clase Principal / Piloto):**

- **Qué hace:** Contiene el método main para probar la Parte 2. Crea los personajes (Aldeano, Brakus, Lunara y Zafiro) pasándoles valores extremos (como -5 o 150) para probar la cápsula de seguridad, e imprime los ataques para validar las sumas matemáticas en cadena.

Conclusiones.

1. El poder de la reutilización de código (Herencia): La herencia nos demostró cómo podemos evitar escribir el mismo código varias veces. Por ejemplo, al crear al Paladin, no tuvimos que volver a programar su nombre, su vida ni su fuerza, ya que todo eso lo heredó automáticamente de Caballero y Personaje. Esto hace que los programas sean más limpios, cortos y fáciles de mantener.

2. La jerarquía y el uso obligatorio de super: Aprendimos que en Java la construcción de un objeto se hace "de arriba hacia abajo". Un hijo no puede existir si primero no se construye su padre, por eso es obligatorio que la primera línea de un constructor hijo sea super(...). Además, vimos que super no solo sirve para constructores, sino para reutilizar el comportamiento del padre en los métodos (como al hacer return super.atacar() + fuerza;), sumando nuevas características en lugar de reemplazar todo.

3. La seguridad de los datos (Encapsulamiento): A pesar de que las clases hijas heredan los atributos de las clases padres, el hecho de que sean private significa que ni siquiera los hijos pueden tocarlos directamente. Esto nos obligó a usar los métodos getter o super.toString(). Aunque parezca un obstáculo, en realidad es una medida de seguridad vital: garantiza que reglas como "la vida no puede ser menor a 0 ni mayor a 100" (definida en el padre) no puedan ser violadas por ninguna clase hija.

4. Organización estricta de Java: Los errores de compilación que resolvimos dejaron una lección muy clara sobre la estructura de Java: cada clase pública debe vivir en su propio archivo .java con el nombre exacto de la clase. Esta regla obliga a los programadores a mantener sus proyectos ordenados, lo cual es fundamental cuando se trabaja en sistemas grandes.